

Philosophers Project

Low-Level OS, Threads & Concurrency Roadmap

A structured study path from CPU fundamentals to advanced concurrency and Linux internals.

Phase 1: CPU Fundamentals

- Registers (RIP, RSP, general registers)
- Fetch, Decode, Execute cycle
- Function calls and stack frames
- Process memory layout (Text, Data, Heap, Stack)
- Recursion, malloc, memory regions

Phase 2: Processes

- What a process is
- PID, memory, files, CPU state
- task_struct concept
- fork(), execve(), wait()
- ps, top, htop

Phase 3: Why Threads Exist

- Single execution flow vs multiple execution flows
- Shared resources: code, heap, globals, files
- Private resources: stack, registers, instruction pointer

Phase 4: What a Thread Really Is

- Registers
- Instruction Pointer
- Stack
- Kernel scheduling

Phase 5: Context Switching

- Saving/restoring registers
- Timer interrupts
- Switching execution between threads

Phase 6: Scheduling

- Running, Ready, Blocked states
- Time slices
- Preemption

- Round Robin
- Linux CFS

Phase 7: Race Conditions

- Read-modify-write operations
- Why counter++ is not atomic
- Data corruption examples

Phase 8: Mutexes

- Mutual Exclusion
- Critical sections
- `pthread_mutex_lock()`
- `pthread_mutex_unlock()`

Phase 9: Mutex Internals

- Atomic operations
- Compare-And-Swap (CAS)
- Futex
- Fast path vs slow path

Phase 10: Memory Visibility

- CPU caches
- Cache coherence
- Memory ordering
- Memory barriers

Phase 11: Condition Variables

- Busy waiting vs sleeping
- `pthread_cond_wait()`
- `pthread_cond_signal()`
- `pthread_cond_broadcast()`

Phase 12: Why Condition Variables Need Mutexes

- Lost wakeup problem
- Atomic check-and-sleep

- Correct waiting patterns

Phase 13: Deadlocks

- Mutual exclusion
- Hold and wait
- No preemption
- Circular wait
- Dining Philosophers deadlock

Phase 14: Linux Internals

- Processes
- Threads
- Scheduler
- Futexes
- Virtual memory
- Recommended Reading: OSTEP, Modern Operating Systems, Linux Kernel Development

Phase 15: After Philo

- CPU architecture
- Operating systems
- Linux scheduler
- Memory models
- Atomics
- Lock-free programming
- Linux kernel source code